



CCODEL
RAISING A NEW GENERATION OF
GLOBAL LEADERS

Computer Programming I (Java 1)

COS221

Mr. Otavie Okuoyo

Phd (in view Computer Science)

College of Science and Technology



Visit Our Website

ccodel.covenantuniversity.edu.ng



MODULE SEVEN: STRINGS AND STRING PROCESSING

UNIT ONE: STRINGS AND STRING PROCESSING

STRINGS AND STRING PROCESSING

Learning Objectives

By the end of this unit, you will be able to:

- **explain the concept of strings in Java.**
- **perform common string operations such as concatenation, substring extraction, and searching.**
- **differentiate between String, StringBuilder, and StringBuffer.**
- **apply StringBuilder and StringBuffer for efficient string manipulation.**

Learning Outcomes

At the end of this unit, you should be able to:

- **understand the concept of strings in Java.**
- **perform common string operations (concatenation, substring, searching, etc.).**
- **differentiate between String, StringBuilder and StringBuffer.**
- **use StringBuilder and StringBuffer for efficient string manipulation.**

Introduction

- **Strings are one of the most commonly used data types in programming.**
- **They represent sequences of characters and are essential for storing and manipulating text data. This lesson covers string manipulation techniques in Java and explores classes like *String*, *StringBuilder*, and *StringBuffer* for efficient string processing.**

What is a String?

- A String is an object representing a sequence of characters (chars values) enclosed within a **double quotes (“”)**. **String** is a final class in **java.lang** package. That is, **String** class is final, meaning it cannot be subclassed.
- Strings can be created using:
 - i. String Literals: Stored in String Pool for memory efficiency.
 - ii. The **new** Keyword: Creates a new object in Heap Memory.

Creating a String?

Examples of String Creation Using String Literals:

- ***String*** s1 = ***“Hello”***;
- ***String*** s2 = ***“Hello”***; // Reuses the same object from String Pool

Examples of String Creation Using the new Keyword:

- ***String*** s3 = new **String**(***“Hello”***);
- ***String*** s4 = new **String**(***“Hello”***); // Creates a new object

String Immutability

- Strings are immutable, meaning once created, their values cannot be changed. Any operation to modify a string will actually create a new String object. Example:

```
public class StringConcatExample {  
    public static void main(String[] args) {  
        String str = "Java";  
        str.concat(" Programming");  
        System.out.println("After concat without assignment: " + str);  
        String newStr = str.concat(" Programming");  
        System.out.println("After concat with assignment: " + newStr);  
    }  
}
```

- The output of the code is as follows:

After concat without assignment: Java

After concat with assignment: Java Programming

Common String Methods

String Concatenation:

- String concatenation can occur using the **+** operator or the *concat()* method as shown in the code snippets below.

```
String firstName = "John";
```

```
String lastName = "Doe";
```

```
String fullName = firstName + " " + lastName;
```

```
String result = "Hello".concat(" World");
```

```
System.out.println(fullName); // Output: John Doe
```

```
System.out.println(result); // Output: Hello World
```

Common String Methods (2)

Searching in Strings:

- When working with strings, it is common to search for specific characters, substrings, or patterns within a larger string. Java provides several useful methods in the String class namely *contains()*, *startsWith()*, *endsWith()*, etc.

Example 1: Searching in String Using endsWith()

```
String str = "Hello.java";
```

```
System.out.println(str.endsWith(".java")); // Output: true
```

```
System.out.println(str.endsWith("Java")); // Output: false
```

Common String Methods (3)

Searching in Strings (Continuation):

Example 2: Searching in String Using indexOf()

String str = "Hello, World!";

System.out.println(str.indexOf('o')); // Output: 4

System.out.println(str.indexOf("World")); // Output: 7

Example 3: Searching in String Using contains()

String str = "Hello, World!";

System.out.println(str.contains("World")); // Output: true

System.out.println(str.contains("world")); // Output: false

Common String Methods (4)

String Comparison

- We use *equals()* (case-sensitive) vs *equalsIgnoreCase()*. *compareTo()* is used for lexicographical comparison.

String a = "Java"; String b = "java";

System.out.println(a.equals(b)); // Output: false

*System.out.println(a.equalsIgnoreCase(**b**)); // Output: true*

*System.out.println(a.compareTo(**b**)); // Output: -32*

- In the comparison *a.compareTo(**b**)*, '***J***' (Unicode value of ***74***) is lexicographically less than '***j***' (Unicode value of ***106***)

Common String Methods (5)

Extracting Substring

- We use substring (int start) and substring (int start, int end) to extract a portion of the string. Start is inclusive while end is exclusive. Given the code snippet below:

```
String str = "Covenant University";  
System.out.println(str.substring(3));  
System.out.println(str.substring(0, 4));
```

- The output is as follows:

```
enant University  
Cove
```

StringBuilder and StringBuffer

The String class is immutable. Every modification creates a new String object, which can be inefficient when dealing with frequent or large-scale string operations. To overcome this, Java provides two mutable alternatives:

- i. StringBuilder**
- ii. StringBuffer**

StringBuilder

- **StringBuilder is a mutable sequence of characters introduced in Java 5. It is not synchronized, meaning it is not thread-safe, but it offers better performance in single-threaded environments.**
- **The following are common methods used with StringBuilder: `append(String str)`, `insert(int offset, String str)`, `replace(int start, int end, String str)`, `delete(int start, int end)`, ***`reverse()`***, ***`toString()`***, etc. Consider the code block below:**

StringBuilder (2)

```
StringBuilder sb = new StringBuilder("Java");  
sb.append(" Programming");  
sb.insert(5, "Language");  
sb.replace(5, 13, "Core");  
sb.delete(5, 10);  
sb.reverse();  
System.out.println("Result: " + sb);
```

The output of the code is Result: gnimmargorP avaJ

StringBuffer

StringBuffer is similar to ***StringBuilder*** but is synchronized. This makes it suitable for multi-threaded environments where multiple threads may access the same string data. Methods are similar to those in ***StringBuilder***.

String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread-safe	No	No	Yes
Performance	Slow (many modification)	Fast (single-threaded)	Slower (due to synchronization)
Use Case	When content does not change.	For fast operations in single thread.	For fast operations in multi-threading.

In-Text Questions (ITQs)

- 1. What is a String, and how can you create one?**
- 2. What is the difference between creating a String using a literal and using the new keyword?**
- 3. Why are Strings considered immutable in Java, and what is the implication of this?**
- 4. When should you use StringBuilder instead of String?**

In-Text Answers (ITAs)

1. A String is an object that represents a sequence of characters enclosed in double quotes. It is a final class in the java.lang package. Strings can be created in two ways:

Using String literals: `String s1 = "Hello";`

Using the new keyword: `String s2 = new String("Hello");`

2. Creating a String using a literal (e.g., `String s1 = "Hello";`) stores the string in the String Pool, allowing memory reuse. Using the new keyword (e.g., `String s3 = new String("Hello");`) creates a new object in the heap memory, even if an identical string exists.

In-Text Answers (ITAs)

- 3. Strings are immutable because once created, their values cannot be changed. Any modification (like concatenation) results in a new String object being created. This improves security and performance but can be inefficient for frequent changes.**
- 4. Use StringBuilder when performing frequent string modifications (e.g., concatenation, insertion, deletion) in a single-threaded environment, as it is mutable and avoids creating multiple objects like String.**

Conclusion

- **This unit introduced key concepts of string handling in Java, emphasizing that strings are immutable sequences of characters. You learned to create strings using literals and the new keyword, and explored common operations like substring extraction, searching, and comparison.**
- **It also highlighted the limitations of immutable strings for frequent modifications and presented StringBuilder and StringBuffer as efficient, mutable alternatives for single-threaded and multi-threaded environments, respectively.**

Summary

- **Strings in Java are immutable sequences of characters created via literals or the new keyword.**
- **Common methods like `substring()`, `charAt()`, `split()`, `contains()`, and `indexOf()` allow effective text manipulation and searching.**
- **For comparison, `equals()` and `compareTo()` perform case-sensitive and lexicographical checks.**
- **To handle frequent modifications efficiently, `StringBuilder` (faster, non-synchronized) and `StringBuffer` (thread-safe, synchronized) offer mutable alternatives based on concurrency needs.**